

Taller: Rendimiento en Aplicaciones Web - Catálogo de Productos

El rendimiento en aplicaciones web se mide principalmente por:

- Tiempo de Respuesta: Milisegundos desde request hasta response
- Throughput: Requests Per Second (RPS) que puede manejar
- Escalabilidad: Capacidad de mantener rendimiento bajo demanda creciente
- Utilización de Recursos: CPU, memoria, conexiones DB, hilos

Factores Críticos de Rendimiento

1. Base de Datos: Consultas, índices, diseño de esquema
2. Cache: Reducción de accesos a DB
3. Concurrencia: Manejo de múltiples requests simultáneos
4. I/O: Operaciones de red y disco

Percentiles de latencia. El percentil pxx es el tiempo por debajo del cual cae $xx\%$ de las respuestas. Por ejemplo, **p95 = 250 ms** significa que **el 95%** de las respuestas tardan **250 ms o menos**.

Contexto - Sistema de Catálogo de Productos

Una empresa de e-commerce lo ha contratado para realizar un sistema que permita fácilmente consultar los productos disponibles en el inventario. Los productos están organizados por categoría.

La empresa ha adelantado algunas conversaciones con los stakeholders más relevantes y esto es lo que han indicado:

☐ **“La estudiante Valentina en hora pico”**

En el almuerzo, Valentina busca *laptops* en la categoría “Electrónica”, ajusta por precio y revisa el inventario del modelo que le gusta. La demanda es alta, pero ella espera resultados en menos de 400ms.

☐ **“Camila, analista de ventas, al cierre del mes”**

Camila entra al módulo de *Top Selling* para consolidar tendencias mensuales; espera tiempos de respuesta de menos de 700ms aun con consultas pesadas.

☐ **“Óscar en bodega”**

Óscar actualiza inventarios tras la recepción de nueva mercancía; el sistema debe reflejar en menos de 400ms el nuevo stock.

Realice el análisis de lo que dicen los stakeholders y construya una aplicación web para un e-commerce. El back debe exponer APIs REST para mínimo las siguientes

funcionalidades, trate de aplicar principios de diseño como Responsabilidad Única, Segregación de Interfaces e Inversión de Dependencias para el diseño de las APIs. Debe documentar el API usando OpenAPI (use springdoc-openapi-starter-webmvc-api).

- Búsqueda de productos: Por categoría, precio, nombre
- Detalles de producto: Información completa incluyendo inventario, precio, descripción, comentarios, calificaciones
- Gestión de inventario: Actualizaciones de stock
- Reportes: productos más vendidos

Tenga en cuenta para el taller:

- Tecnologías Base para los experimentos. Recuerde, no debe iniciar con el uso de todas las tecnologías. El primer paso es hacerlo con lo que conoce actualmente, posteriormente se aplican los cambios dependiendo del experimento. De otra forma su entrega no será válida y la nota será 0.0.
 - Java 17+
 - Spring Boot 3.x
 - Spring WebFlux + R2DBC
 - PostgreSQL 15 con pg_stat_statements
 - Redis 7.x para cache
 - JMeter 5.x para load testing
 - Docker + Docker Compose (opcional)
 - Herramientas de Monitoreo
 - Application: Spring Boot Actuator + Micrometer
- Debe cargar 70,000 productos con categorías, precios, descripciones.
 - Puede apoyarse en IA para poblar la base de datos.
 - 10 categorías principales con subcategorías
 - 100,000 transacciones de inventario históricas. (70% ventas, 20% entradas, 5% devoluciones, 5% ajustes(aumento o disminución de inventario sin ser ventas o compras))
 - 1,000 usuarios simulados para las pruebas

Línea Base: Construya el API REST, tal como lo hace hoy en día, con los siguientes endpoints para el contexto indicado. Como mínimo:

GET /api/products/search?query=laptop&category=electronics&minPrice=500

GET /api/products/{id}/details

GET /api/reports/top-selling?period=month

GET /api/inventory/low-stock

Realice algunas pruebas para verificar el comportamiento inicial de la aplicación. Posteriormente, realice los siguientes experimentos, analice y documente como las diferentes modificaciones mejoran el rendimiento de la aplicación. Para estas

pruebas, realice el procedimiento descrito con JMeter más abajo en este documento.

Experimento 1: Optimización de Base de Datos

Objetivo: Medir el impacto de índices, vistas y optimización de consultas.

Escenarios a Implementar:

1.1 Sin Optimización

- Consultas directas sin índices
- Búsqueda por texto usando LIKE '%producto%'
- Joins sin optimizar (opcional, si hay lugar a JOINS)

1.2 Con Índices Estratégicos

- Índices en campos de búsqueda frecuente
- Índices compuestos para filtros combinados
- Índices parciales para consultas específicas

1.3 Consultas Optimizadas

- Paginación

Métricas:

- Tiempo de ejecución de consultas SQL
- Tiempo de respuesta de APIs (ms)
- Peticiones por segundo (cantidad máxima soportada)

Experimento 2: Implementación de Cache

Objetivo: Evaluar diferentes estrategias de caching en rendimiento.

Estrategias de Cache:

2.1 Sin Cache

- Todas las consultas van directo a la base de datos

2.2 Cache de Aplicación

- Caffeine Cache en memoria
- TTL de 5 minutos para productos

2.3 Cache Distribuido (Redis)

- Redis como cache externo
- Serialización JSON de objetos
- TTL configurable por tipo de dato

2.4 Cache Multi-Nivel

- L1: Cache local (Caffeine)

- L2: Cache distribuido (Redis)
- Estrategia de write-through y write-behind

Experimento 3: Programación Tradicional vs Reactiva

Objetivo: Comparar Spring Boot tradicional vs Spring WebFlux reactivo.

3.1 Spring Boot Tradicional

- Controllers bloqueantes
- JPA con Hibernate
- ThreadPool tradicional (200 hilos)
- Conexiones DB síncronas

3.2 Spring WebFlux Reactivo

- Controllers no-bloqueantes con Mono/Flux
- R2DBC para base de datos reactiva

JMeter - Configuración de Tests

Escenarios de Carga:

- Carga Baja: 100 usuarios concurrentes
- Carga Media: 500 usuarios concurrentes
- Carga Alta: 1000 usuarios concurrentes
- Carga Extrema: 5000 usuarios concurrentes

Test Plan para línea base y para cada experimento

- Thread Groups: 1, 10, 50, 100, 200, 500, 700, 1000, 1500, 5000 usuarios
- Ramp-up period: 60 segundos
- Duration: 300 segundos (5 minutos)
- Think time: 1-3 segundos entre requests

Métricas:

- Response Time: Average, Median, 90th, 95th, 99th percentile
- Throughput: Requests/second, Transactions/minute
- Error Rate: Porcentaje de requests fallidos

Entregable: Reporte de Experimentación

Entregue un documento PDF con el análisis del contexto: Diagrama de casos de uso, Diagrama de componentes, Escenarios de calidad.

Adicionalmente, indique los cambios en código y los resultados obtenidos en cada experimentación, detalle los cambios realizados y los resultados obtenidos en las pruebas con JMeter para cada cambio.

Adicionalmente:

Código Fuente

- Código reactivo y no reactivo, es decir código con webflux y sin webflux
- Scripts SQL para creación de índices y vistas
- Configuraciones de JMeter (.jmx files)
- Docker Compose para reproducir el entorno

Datos de Inicialización

- Script SQL